

Practical Amortized Bootstrapping for NTRU-Based FHE

Wun-Ting Lin and Ja-Ling Wu

National Taiwan University, Taipei, Taiwan

d07922020@ntu.edu.tw, wjl@cmlab.csie.ntu.edu.tw

Abstract. Fully homomorphic encryption (FHE) enables computation on encrypted data. NTRU-based FHE offers compact ciphertexts—a single ring element rather than a tuple—but FINAL, the leading NTRU-based scheme, represents its ciphertexts and decryption in matrix form that hides the polynomial-ring structure on which recent automorphism-based amortized bootstrapping relies.

We reformulate FINAL in the standard polynomial-ring setting and express its decryption as coefficient-wise inner products compatible with FHEW-style accumulators. This preserves the required ring automorphisms and lets us adapt monomial-by-polynomial amortized bootstrapping to NTRU ciphertexts with sparse secret keys: for a secret of Hamming weight h , the dominant per-coefficient work drops from $O(n\ell_Q)$ to $O(h\ell_{\text{pos}})$ external-product operations, where ℓ_Q and ℓ_{pos} denote the gadget-decomposition lengths of the standard and position-based bootstrapping keys, respectively. Concrete parameters are selected by a joint optimization that combines a refined average-case noise analysis with security validation via NTRU fatigue analysis and the Lattice Estimator under sparse-secret distributions.

Our highly optimized single-threaded C++ implementation based on Intel HEXL bootstraps a message coefficient in 2.68 ms at $n = 8192$ — $45\times$ faster than FINAL and $2.6\times$ faster than TFHE-rs. The bootstrapping key does not scale with n (11.28 MB at $n = 8192$). Packed ciphertexts cost about 8 bytes per message bit, and the total key material remains below that of state-of-the-art amortized RLWE bootstrapping.

Keywords: Fully Homomorphic Encryption · NTRU · Amortized Bootstrapping

1 Introduction

Fully homomorphic encryption (FHE) enables computation directly on encrypted data, but bootstrapping remains a major performance bottleneck—particularly in FHEW-style bitwise schemes, where it refreshes ciphertexts and evaluates nonlinear operations. Therefore, reducing the cost and memory footprint of bootstrapping is therefore a central challenge in designing practical FHE schemes.

NTRU-based FHE is attractive because its ciphertexts are compact: a single ring element rather than a tuple of them. Early constructions such as LTV [LTV12] and BLLN [BLLN13] were later broken in the overstretched parameter regime [ABD16, KF17], but Ducas and van Woerden showed how to avoid this regime [DvW21], and Bonte et al. [BIP⁺22] subsequently demonstrated FINAL, an NTRU-based FHE scheme with competitive bootstrapping performance under conservative parameters.

The key idea behind FINAL is a matrix-NTRU formulation that expresses decryption as an inner product, which can be evaluated homomorphically inside an FHEW-style accumulator. This advantage comes at a structural cost: monomial-by-polynomial (MPMul) amortized bootstrapping packs n messages into a single accumulator polynomial and traverses the secret key through monomial multiplications and Galois automorphisms—inherently

ring operations with no natural counterpart once ciphertexts are treated as matrices and vectors over $\mathbb{Z}_q$. FINAL therefore retains compact NTRU ciphertexts but cannot directly benefit from automorphism-based amortization.

Can we obtain an NTRU-based FHE scheme that simultaneously preserves compact NTRU ciphertexts, supports FHEW-style bootstrapping, and enables amortized bootstrapping via polynomial automorphisms? Returning to the standard polynomial formulation of NTRU [HPS98], we decompose the decryption product into coefficient-wise inner products. This decomposition plays the same role as FINAL’s matrix lifting—making decryption compatible with FHEW-style accumulators—while operating natively within the ring. This ensures that ciphertexts and secret keys are kept as ring elements, leaving the automorphisms required for amortization fully accessible.

Building on this observation, we adapt the MPMul-based amortized bootstrapping to NTRU ciphertext. Guimarães and Pereira [GP25] originally developed this technique for RLWE ciphertext. The adaptation is nontrivial—NTRU accumulator ciphertexts are scalar ring elements of the form g/f rather than RLWE tuples, and accumulator operations must preserve this structure—and is achieved through a difference-vector representation of the sparse secret-key positions and an NTRU-compatible MPMul procedure for accumulator ciphertexts. For a sparse ternary secret of Hamming weight h , the procedure traverses only the h nonzero key positions instead of all n , reducing the amortized cost per refreshed coefficient from $O(n\ell_Q)$ to $O(h\ell_{\text{pos}})$ —where $\ell_{\text{pos}} = O(\log R_{\text{pos}})$ is the short gadget length induced by the bound R_{pos} on the gaps between nonzero positions—and shrinking the dominant bootstrapping-key material to sparse position-dependent keys. We further shorten all gadget lengths via an approximate gadget decomposition, and derive concrete parameters via a joint optimization that combines a refined average-case noise analysis with a failure-probability target of 2^{-64} ; security is validated against the NTRU fatigue analysis and the Lattice Estimator for sparse secrets.

We benchmark an optimized C++ implementation, accelerated with Intel HEXL and executed single-threaded on identical hardware, against the reference implementation of FINAL [BIP⁺22], the highly optimized TFHE-rs library [Zam22], and the amortized bootstrapping of Guimarães and Pereira [GP25]. The amortized cost per refreshed message decreases monotonically with the packing dimension, from 5.02 ms at $n = 1024$ to 2.68 ms at $n = 8192$ —speed-ups of $24.14\times$ to $45.22\times$ over FINAL and of $1.41\times$ to $2.65\times$ over TFHE-rs—while staying within $1.35\times$ ($n = 2048$) to $1.17\times$ ($n = 8192$) of GP25, with the gap narrowing as n grows. Our key material is strictly smaller: the bootstrapping key does not scale with n and shrinks to 11.28 MB at $n = 8192$ — $7.7\times$ below FINAL’s and consistently below GP25’s—the total key size is up to $1.98\times$ smaller than GP25’s, and packed ciphertexts cost about 8 bytes per message bit, half of GP25’s per-slot footprint and roughly a $400\times$ density improvement over single-bit FHE schemes.

In summary, our main contributions are as follows:

- **An accumulator-compatible reformulation of NTRU decryption** as coefficient-wise inner products, showing that FHEW-style bootstrapping is directly compatible with the standard polynomial formulation of NTRU—without FINAL’s matrix lifting, and while preserving the ring automorphisms required for amortization.
- **An amortized bootstrapping procedure for NTRU ciphertexts with sparse ternary keys**, combining an NTRU-compatible MPMul with a difference-vector encoding of secret-key positions, reducing the per-coefficient cost from $O(n\ell_Q)$ to $O(h\ell_{\text{pos}})$ and correspondingly shrinks the bootstrapping key.
- **A refined noise analysis and joint parameter optimization** with approximate gadget decompositions under a failure-probability target. Security is validated using NTRU fatigue analysis and the Lattice Estimator, together with an optimized

C++/HEXL implementation evaluated single-threaded against FINAL, TFHE-rs, and GP25 on identical hardware.

2 Preliminaries

2.1 Notation

Throughout this paper, we denote the security parameter by λ . For an integer $d \geq 1$, we write $[d] := \{0, 1, \dots, d-1\}$. Let $n = 2^k$ and $N = 2^K$ for some positive integers k and K . We use n for the dimension of the base ring and N for the dimension of the accumulator ring. We define the cyclotomic rings $\mathcal{R} := \mathbb{Z}[X]/\langle X^n + 1 \rangle$ and $\widehat{\mathcal{R}} := \mathbb{Z}[X]/\langle X^N + 1 \rangle$. For moduli q and Q , their corresponding quotient rings are $\mathcal{R}_q := \mathcal{R}/q\mathcal{R} \cong \mathbb{Z}_q[X]/\langle X^n + 1 \rangle$ and $\widehat{\mathcal{R}}_Q := \widehat{\mathcal{R}}/Q\widehat{\mathcal{R}} \cong \mathbb{Z}_Q[X]/\langle X^N + 1 \rangle$. The modulus q is used by the base encryption scheme, whereas Q is used by the accumulator scheme.

For a modulus μ , let $\mathcal{I}_\mu := [-\lfloor \mu/2 \rfloor, \lceil \mu/2 \rceil] \cap \mathbb{Z}$ and denote by $[z]_\mu^c$ the centered representative of $z \in \mathbb{Z}_\mu$ in $\mathcal{I}_\mu$. Centered lifting is applied coefficient-wise to vectors and polynomials. Vectors are written in bold lowercase and polynomials in italic lowercase. For $a(X) = \sum_{i=0}^{n-1} a_i X^i \in \mathcal{R}_q$, we write $\mathbf{a} := (a_0, \dots, a_{n-1})^T$ and $\tilde{\mathbf{a}} := ([a_0]_q^c, \dots, [a_{n-1}]_q^c)^T$ and define $\|a\|_\infty := \|\tilde{\mathbf{a}}\|_\infty$. The same conventions apply to $\widehat{\mathcal{R}}_Q$ with (n, q) replaced by (N, Q) . We freely identify a polynomial with its coefficient vector when no confusion arises.

For polynomials a, b of dimension d , their coefficient-wise product and inner product are respectively represented as $a \circ b := \sum_{i=0}^{d-1} a_i b_i X^i$ and $\langle a, b \rangle := \sum_{i=0}^{d-1} a_i b_i$. We write $a^{\circ 2} := a \circ a$. In particular, for binary polynomials a and b , $(a - b)^{\circ 2} = a + b - 2(a \circ b)$.

We denote the set of signed monomials by $\mathbb{M} := \{\pm b \cdot X^k : b \in \{0, 1\}, k \in \mathbb{N}\}$.

2.2 Gadget Decomposition and Security Assumptions

Gadget decomposition. We describe gadget decomposition uniformly for the base and accumulator rings. Let $\mathcal{S}_\mu := \mathbb{Z}_\mu[X]/\langle X^d + 1 \rangle$, where (d, μ) is either (n, q) or (N, Q) . Fix a power-of-two decomposition base $B \geq 2$, and let $\ell_\mu := \lceil \log_B \mu \rceil$ denote the number of digits. Since B is even, $\mathcal{D}_B = [-B/2, B/2] \cap \mathbb{Z}$, the balanced digit set, is not symmetric about zero; this asymmetry affects the second moment of the digit distribution and is accounted for in the Section 5.4 noise analysis.

We define the *gadget vector* $\mathbf{g}_\mu := (B^0, B^1, \dots, B^{\ell_\mu-1})^T \in \mathbb{Z}_\mu^{\ell_\mu}$. For any $k \in \mathbb{Z}_\mu$, we define its *base- B decomposition* as $\mathbf{g}_\mu^{-1}(k) := (k_0, k_1, \dots, k_{\ell_\mu-1})^T \in \mathcal{D}_B^{\ell_\mu}$ for all i , satisfying $\langle \mathbf{g}_\mu^{-1}(k), \mathbf{g}_\mu \rangle \equiv k \pmod{\mu}$. For a polynomial $a = \sum_{i=0}^{d-1} a_i X^i \in \mathcal{S}_\mu$, we extend the decomposition coefficient-wise: $\mathbf{g}_\mu^{-1}(a) := \sum_{i=0}^{d-1} \mathbf{g}_\mu^{-1}(a_i) \cdot X^i \in \mathcal{S}_\mu^{\ell_\mu}$. Taking the inner product with $\mathbf{g}_\mu$ reconstructs the polynomial in $\mathcal{S}_\mu$:

$$\langle \mathbf{g}_\mu^{-1}(a), \mathbf{g}_\mu \rangle = \sum_{i=0}^{d-1} a_i \cdot X^i = a.$$

We omit the modulus subscript when clear from context, writing $\mathbf{g}$ and $\mathbf{g}^{-1}(\cdot)$.

We also use the *approximate* gadget decomposition [CGGI16], which retains only the $\ell \leq \ell_\mu$ most significant digits. Let $\varepsilon := \ell_\mu - \ell$. The gadget vector is $\mathbf{g}_{\mu, \ell} := (B^\varepsilon, \dots, B^{\ell_\mu-1})^T \in \mathbb{Z}_\mu^\ell$, and reconstruction holds up to a rounding error: $\langle \mathbf{g}_{\mu, \ell}^{-1}(a), \mathbf{g}_{\mu, \ell} \rangle = a - e_{\text{dec}}(a)$ in $\mathcal{S}_\mu$, with $\|e_{\text{dec}}(a)\|_\infty \leq B^\varepsilon/2$.

NTRU assumption. The security of our scheme relies on the standard NTRU assumption [HPS98]: for f, g sampled from a small-coefficient distribution χ_σ over $\mathcal{R}$ with f invertible in $\mathcal{R}_q$, the public key $h := g \cdot f^{-1} \in \mathcal{R}_q$ is computationally indistinguishable

from a uniformly random element of $\mathcal{R}_q$. We assume parameters are chosen outside the overstretched regime [ABD16, KF17]; the concrete security evaluation, including the secret key’s sparsity, is presented in Section 5.

2.3 FHEW-Style Bootstrapping

FHEW-style bootstrapping [DM15, CGGI16] refreshes a noisy ciphertext $\text{ct} \in \mathcal{R}_q$ by homomorphically evaluating its decryption inside an accumulator: ct is lifted into an accumulator ciphertext over $\hat{\mathcal{R}}_Q$ initialized with a test polynomial, the accumulator is blind-rotated using a bootstrapping key bsk . Because bsk encrypts the base secret key under the accumulator scheme, the refreshed message is extracted—typically as the constant term of the resulting polynomial—and modulus- and key-switching convert the result back to the base key. We follow this paradigm, but instantiate both the base and the accumulator scheme with NTRU ciphertexts; the realization of blind rotation via NTRU external products is presented in Section 3.3.

3 Polynomial NTRU FHE Scheme

FINAL [BIP⁺22] expresses decryption as an inner product between a ciphertext and the first column $\text{col}_0(\mathbf{F}^{-1})$ of the inverse secret-key matrix. This representation is well suited to accumulator-based bootstrapping, but Galois automorphisms and monomial shifts—the operations on which MPMul-based amortization relies—have no natural counterpart in this matrix–vector form. We therefore restate the relevant NTRU operations directly over the polynomial ring; the reformulation preserves the decryption structure required by the accumulator while supporting the automorphism and monomial interface used by the amortized procedure of Section 4.

3.1 Polynomial-Ring NTRU Base Scheme

We now describe the polynomial-ring NTRU base scheme used in our construction. The scheme follows the NTRU encryption structure underlying FINAL [BIP⁺22] and represents ciphertexts and secret keys directly as elements of the polynomial ring. All operations take place in the cyclotomic ring $\mathcal{R}_q$ introduced in Section 2.1. Fresh ciphertexts encode a message in $\mathbb{Z}_4$ and use the scaling factor $\Delta := \lfloor q/4 \rfloor$, whereas the outputs of homomorphic NAND gates carry a message in $\mathbb{Z}_2$ and therefore use $\Delta := \lfloor q/2 \rfloor$.

- **NTRU.ParamGen**(1^λ): On input the security parameter λ , output a parameter tuple (n, q, σ) .
- **NTRU.KeyGen**(n, q, σ): Sample $f' \leftarrow \chi_\sigma^n$ and set $f := 1 + 4f'$; resample f' until f is invertible in $\mathcal{R}_q$. Sample a ternary polynomial $g_{\text{evk}} \in \mathcal{R}_q$ and define the evaluation key $\text{evk} := (g_{\text{evk}} + \lfloor 5q/8 \rfloor) \cdot f^{-1} \in \mathcal{R}_q$. Output (sk, evk) with $\text{sk} := f$.
- **NTRU.Enc**(m, sk): Given a binary polynomial $m = \sum_{i=0}^{n-1} m_i X^i \in \mathcal{R}_q$ with $m_i \in \{0, 1\}$, sample $g \leftarrow \chi_\sigma^n$ and output

$$c := (g + \Delta \cdot m) \cdot f^{-1} \in \mathcal{R}_q.$$

- **NTRU.Dec**(c, sk): Compute $r := c \cdot f \pmod{q}$ and output $m' := \lfloor 2r/q \rfloor \pmod{2}$, where rounding and reduction are applied coefficient-wise.

Lemma 1 (Decryption Correctness). *Let $c := (g + \Delta \cdot m) \cdot f^{-1} \in \mathcal{R}_q$, which encrypts a binary polynomial $m = \sum_{i=0}^{n-1} m_i X^i$ with $m_i \in \{0, 1\}$ at scale $\Delta = \lfloor q/2 \rfloor$. If the centered error satisfies $\|e\|_\infty < \Delta/2$, then $\text{NTRU.Dec}(c, \text{sk}) = m$.*

The proof of Lemma 1 is deferred to Appendix A.

Table 1: Ciphertext formats in the accumulator scheme.

Format	Encrypts	Used for
EncS	accumulator state / test vector	bootstrapping state
EncV	signed monomial in $\mathbb{M}$ or zero	external product / CMux

Homomorphic NAND For binary polynomials $a, b \in \mathcal{R}_q$, the target NAND polynomial is defined coefficient-wise as

$$y := \text{NAND}(a, b) = 1 - a \circ b, \text{ with } y_i = 1 - a_i b_i,$$

where $1 \in \mathcal{R}_q$ denotes the all-ones polynomial $\sum_{i=0}^{n-1} X^i$ in this Hadamard context.

- $\text{NTRU.Nand}(c_0, c_1, \text{evk})$: Given fresh encryptions $c_0 = (g_0 + \Delta \cdot a) \cdot f^{-1}$ and $c_1 = (g_1 + \Delta \cdot b) \cdot f^{-1}$ of binary polynomials $a, b \in \mathcal{R}_q$, output $c_{\text{NAND}} := \text{evk} - c_0 - c_1$.

Lemma 2 (Correctness of NTRU.Nand). *Let c_0 and c_1 be fresh encryptions of binary polynomials a and b as above, and let evk be generated from the ternary polynomial g_{evk} . Denote by $\mathbf{g}_0, \mathbf{g}_1 \in \mathbb{Z}_q^n$ the coefficient vectors of g_0 and g_1 , respectively. If $|(\mathbf{g}_0)_i + (\mathbf{g}_1)_i| < (q - 20)/8$ for every $i \in \{0, \dots, n-1\}$, then $\text{NTRU.Nand}(c_0, c_1, \text{evk})$ outputs a valid binary-output ciphertext of $y = \text{NAND}(a, b) = 1 - a \circ b$.*

The proof of Lemma 2 is deferred to Appendix B.

3.2 Accumulator-Compatible Decryption Reformulation

Decryption as Coefficient-Wise Inner Products The decryption step described above requires a polynomial multiplication of the ciphertext by the secret key. To evaluate this operation homomorphically using an FHEW-style accumulator, we rewrite the polynomial multiplication as a set of inner products, one for each output coefficient. Let $a(X) = \sum_{i=0}^{n-1} a_i \cdot X^i$ and $f(X) = \sum_{i=0}^{n-1} f_i \cdot X^i$ be two polynomials in $\mathcal{R}_q = \mathbb{Z}_q[X]/\langle X^n + 1 \rangle$. Their negacyclic product is $c(X) = a(X) \cdot f(X) = \sum_{k=0}^{n-1} c_k X^k$. For each coefficient index $0 \leq k < n$, the coefficient c_k is given by $c_k = \sum_{i=0}^k a_{k-i} \cdot f_i - \sum_{i=k+1}^{n-1} a_{n+k-i} \cdot f_i$.

Equivalently, c_k can be expressed as the inner product of the coefficient vectors of $a(X^{-1}) \cdot X^k$ and the coefficient vector of $f(X)$:

$$c_k = \langle a(X^{-1}) \cdot X^k, f(X) \rangle. \quad (1)$$

This formulation is compatible with FHEW-style accumulators, which compute inner products between public coefficient vectors and encrypted secret-key coefficients.

Hence, we can compute each output coefficient using an accumulator:

$$a(X) \cdot f(X) \mapsto \{ \langle a(X^{-1}) \cdot X^k, f(X) \rangle \}_{0 \leq k < n}.$$

This reformulation provides the ring-automorphism and monomial-operation interface used by the amortized procedure below.

3.3 NTRU-based Accumulator Scheme

Scalar and Vector Accumulator Ciphertexts Building on the basic NTRU scheme in Section 3.1, we now introduce a companion *accumulator scheme* that supports external products, the algebraic primitive underlying accumulator-based bootstrapping. The scheme operates over the larger ring $\hat{\mathcal{R}}_Q$ and provides two ciphertext formats, as shown in Table 1:

EncS encodes a ternary polynomial as a single ring element, whereas **EncV** encodes either a signed monomial or zero via gadget decomposition. Unlike the basic NTRU encryption $(g + \Delta \cdot m) \cdot f^{-1}$, both formats decouple the noise term from the message scaling. This structural choice is motivated by noise-growth considerations: keeping the noise of the form $h \cdot f_Q^{-1}$ separate from $\Delta \cdot u$ yields tighter noise accumulation under repeated external products.

- **ACC.ParamGen**(1^λ): On input the security parameter λ , output a parameter tuple $(N, Q, \varsigma, B, \ell_Q)$, where B denotes the gadget decomposition base and $\ell_Q := \lceil \log_B Q \rceil$.
- **ACC.KeyGen**(N, Q, ς): Sample $f'_Q \leftarrow \chi_\varsigma^N$ and set $f_Q := 1 + 4f'_Q$; resample f'_Q until f_Q is invertible in $\hat{\mathcal{R}}_Q$. Output $\mathbf{sk} := f_Q$.
- **ACC.EncS**($u, \mathbf{sk}$): Given a ternary polynomial $u \in \hat{\mathcal{R}}_Q$, sample $h \leftarrow \chi_\varsigma^N$ and set $\Delta := \lfloor Q/4 \rfloor$. Output the scalar ciphertext

$$c := h \cdot f_Q^{-1} + \Delta \cdot u \in \hat{\mathcal{R}}_Q.$$

- **ACC.EncV**($v, \mathbf{sk}$): Given $v \in \mathbb{M} \cup \{0\}$, define the gadget vector $\mathbf{g} := (B^0, B^1, \dots, B^{\ell_Q-1})^T$ and sample $\mathbf{h} = (h_0, h_1, \dots, h_{\ell_Q-1})^T$ with each $h_i \leftarrow \chi_\varsigma^N$. Output the vector ciphertext

$$\mathbf{c} := \mathbf{h} \cdot f_Q^{-1} + \mathbf{g} \cdot v \in \hat{\mathcal{R}}_Q^{\ell_Q}.$$

External Product. The *external product* between a scalar ciphertext c and a vector ciphertext $\mathbf{c}$ is defined as

$$c_{\text{mult}} := c \boxtimes \mathbf{c} := \mathbf{g}^{-1}(c) \cdot \mathbf{c} \in \hat{\mathcal{R}}_Q,$$

where $\mathbf{g}^{-1}(\cdot)$ denotes the gadget decomposition with respect to $\mathbf{g}$. Since $\mathbf{g}^{-1}(c) \cdot \mathbf{g} = c$, a direct expansion shows that c_{mult} is a valid scalar encryption of the polynomial product $u \cdot v$, provided the resulting noise term h_{mult} remains sufficiently small. The detailed algebraic expansion and correctness proof are deferred to Appendix C.

For the message component to decode correctly, we require $\|u \cdot v\|_\infty < 4$; otherwise, the scaled message $\Delta \cdot (u \cdot v)$ overflows modulo Q , producing an ill-formed ciphertext. This condition holds in our setting. If $v \in \mathbb{M}$, multiplication by v in $\hat{\mathcal{R}}_Q$ is a signed cyclic permutation of the coefficients of u ; if $v = 0$, the product is zero. Hence, $\|u \cdot v\|_\infty \leq \|u\|_\infty \leq 1$.

4 Amortized Bootstrapping for NTRU-Based FHE

4.1 Amortized Bootstrapping via Sparse-Key MPMu1

The MPMu1-based amortization of Guimarães and Pereira [GP25] was developed for RLWE, where ciphertexts are tuples of ring elements; NTRU accumulator ciphertexts are scalar ring elements of the form g/f , so the required monomial operations must be realized while preserving this structure. Beyond the decryption reformulation in Section 3.2, our construction uses three components: sparse-key polynomial multiplication, sign selection for ternary coefficients, and an automorphism key switching step that maps ciphertexts associated with $f_Q(X^{-1})$ back to $f_Q(X)$. A dense coefficient-wise evaluation would require n^2 external accumulator for the n outputs of a packed ciphertext; the amortized construction instead uses h MPMu1 invocations, where h is the Hamming weight of the sparse key.

Sparse-Key Polynomial Multiplication. We first describe, at the plaintext level, how the sparsity of a secret key $f \in \mathcal{R}_q$ accelerates the polynomial product $a \cdot f$ that appears in the homomorphic decryption circuit. Assume f is sparse and ternary, with Hamming weight h , and let $\text{idx}(f) := \{j : f_j \neq 0\}$ with $|\text{idx}(f)| = h$. For an arbitrary polynomial $a \in \mathcal{R}_q$, the product $a \cdot f$ admits the sparse expansion

$$a(X) \cdot f(X) = \sum_{j \in \text{idx}(f)} f_j \cdot a(X) \odot X^j = \sum_{j \in \text{idx}(f)} \text{MPMul}(f_j \cdot a(X), X^j), \quad (2)$$

where $\odot$ denotes monomial-by-polynomial multiplication in $\mathcal{R}_q$, and $\text{MPMul}(p, X^j) := p \odot X^j$ is the algorithmic implementation of this operation in the homomorphic setting. Since $f_j \in \{-1, +1\}$ for every $j \in \text{idx}(f)$, each scalar multiplication $f_j \cdot a(X)$ reduces to a sign choice, so the product costs exactly h monomial-by-polynomial multiplications.

In the next subsection, we lift this decomposition to the ciphertext domain: the polynomial a is encoded as a list of accumulator ciphertexts $\{c_i\}_{i=0}^{n-1}$ in $\hat{\mathcal{R}}_Q$, with each c_i a homomorphic encryption of the monomial X^{a_i} ; the bounded differences between the nonzero positions are provided in encrypted form through the bootstrapping keys; and each MPMul becomes a sequence of external products that homomorphically realizes the list-level negacyclic rotation corresponding to multiplication by X^j .

MPMul over NTRU Accumulator Ciphertexts. We now describe the homomorphic counterpart of the sparse-key multiplication (2). Let $u \in \mathcal{R}_q$ be a message polynomial whose coefficients $u_i \in \mathbb{Z}_q$ are encrypted, for $0 \leq i < n$, as a list of accumulator ciphertexts

$$\mathbf{c} = \{c_i := \text{ACC}.\text{EncS}(X^{u_i}) \mid 0 \leq i < n\}, \quad c_i \in \hat{\mathcal{R}}_Q,$$

that is, each message coefficient is lifted to a monomial in the exponent of an accumulator polynomial in $\hat{\mathcal{R}}_Q$.

Given a shift amount $v \in \mathbb{Z}$ satisfying $0 \leq v < R_{\text{pos}}$, supplied through the bootstrapping keys defined below, the operation $\text{MPMul}(\mathbf{c}, \text{bsk}^{\text{rot}})$ outputs a ciphertext list $\mathbf{c}' = (c'_0, \dots, c'_{n-1})$ encrypting $u(X) \cdot X^v \in \mathcal{R}_q$; that is, $c'_i = \text{ACC}.\text{EncS}(X^{u'_i})$ where $(u'_0, \dots, u'_{n-1})$ is the coefficient vector of $u \odot X^v$.

Decomposition of the shift. Let $\ell_{\text{pos}} := \lceil \log_2 R_{\text{pos}} \rceil$, where R_{pos} upper-bounds the entries of the position-difference vector of the sparse key f (i.e., the consecutive offsets between elements of $\text{idx}(f)$). Write the binary expansion

$$\text{Decomp}(v) = (v_0, v_1, \dots, v_{\ell_{\text{pos}}-1}) \in \{0, 1\}^{\ell_{\text{pos}}}, \quad v = \sum_{i=0}^{\ell_{\text{pos}}-1} v_i \cdot 2^i.$$

The shift then factors into a product of conditional power-of-two shifts: $X^v = \prod_{i=0}^{\ell_{\text{pos}}-1} X^{v_i \cdot 2^i}$.

Power-of-two list rotation. Multiplication of $u \in \mathcal{R}_q$ by X^{2^i} acts on the coefficient vector as a negacyclic rotation by 2^i . At the ciphertext level, this is realized by rearranging the list $\mathbf{c}$ and homomorphically negating the wrapped entries: when $v_i = 1$, the post-shift coefficient vector becomes

$$(u'_0, \dots, u'_{n-1}) = \left(\underbrace{-u_{n-2^i}, \dots, -u_{n-1}}_{2^i \text{ entries, negated}}, \underbrace{u_0, \dots, u_{n-2^i-1}}_{n-2^i \text{ entries}} \right),$$

where the negation of the wrapped entries reflects the relation $X^n = -1$ in $\mathcal{R}_q$ and is implemented homomorphically via the automorphism $X \mapsto X^{-1}$ on the corresponding accumulator ciphertexts. When $v_i = 0$, the list remains unchanged. Iterating this conditional rotation for $i = 0, 1, \dots, \ell_{\text{pos}} - 1$ yields a list that encrypts $u \odot X^v$.

Algorithm 1: Monomial-by-polynomial multiplication

Input: $\mathbf{c} = (c_0, \dots, c_{n-1})$: a list of accumulator ciphertexts with
 $c_t = \text{ACC.EncS}(X^{u_t}) \in \hat{\mathcal{R}}_Q$;
 $\mathbf{bsk}_j^{\text{rot}} = (\mathbf{bsk}_{0,j}^{\text{rot}}, \dots, \mathbf{bsk}_{\ell_{\text{pos}}-1,j}^{\text{rot}})$: bootstrapping keys encoding the binary expansion of
the shift v associated with the j -th nonzero coefficient of f ;
 $\mathbf{ksk}^{\text{aut}}$: key-switching key for the automorphism $X \mapsto X^{-1}$
Output: $\hat{\mathbf{c}} = (\hat{c}_0, \dots, \hat{c}_{n-1})$: a ciphertext list with $\hat{c}_t = \text{ACC.EncS}(X^{(u \odot X^v)_t})$

```

1 for  $0 \leq i < \ell_{\text{pos}}$  do
2   for  $0 \leq t < 2^i$  do
3      $c_{\text{rot}} \leftarrow \text{Automorphism}(c_{n-2^i+t}, -1, \mathbf{ksk}^{\text{aut}}) - c_t$ ;
4      $\hat{c}_t \leftarrow c_{\text{rot}} \boxplus \mathbf{bsk}_{i,j}^{\text{rot}} + c_t$ ;                                     // wrapped
5   for  $2^i \leq t < n$  do
6      $c_{\text{rot}} \leftarrow c_{t-2^i} - c_t$ ;
7      $\hat{c}_t \leftarrow c_{\text{rot}} \boxplus \mathbf{bsk}_{i,j}^{\text{rot}} + c_t$ ;                                     // non-wrapped
8    $\mathbf{c} \leftarrow \hat{\mathbf{c}}$ ;
9 return  $\mathbf{c}$ ;
```

Encoding the shift bits. The bits $\{v_i\}$ are integrated into the scheme via dedicated bootstrapping keys. For each nonzero-position index $j \in \{0, \dots, h-1\}$ and each bit index $i \in \{0, \dots, \ell_{\text{pos}}-1\}$, define

$$\mathbf{bsk}_{i,j}^{\text{rot}} := \begin{cases} \text{ACC.EncV}(0), & v_i^{(j)} = 0, \\ \text{ACC.EncV}(1), & v_i^{(j)} = 1, \end{cases} \quad (3)$$

where $v_i^{(j)}$ denotes the i -th bit of the position difference associated with the j -th nonzero coefficient of f . The full key bundle for a single `MPMul` is $\mathbf{bsk}_j^{\text{rot}} := (\mathbf{bsk}_{0,j}^{\text{rot}}, \dots, \mathbf{bsk}_{\ell_{\text{pos}}-1,j}^{\text{rot}})$. Each $\mathbf{bsk}_{i,j}^{\text{rot}}$ drives, via an external product, the i -th conditional rotation, selecting between the identity ($v_i^{(j)} = 0$) and a rotation by 2^i ($v_i^{(j)} = 1$). The full algorithmic realization of `MPMul` is given in Algorithm 1.

Sign Handling for Ternary Coefficients Let $f = \sum_{i=0}^{n-1} f_i X^i \in \mathcal{R}_q$ be a sparse ternary secret key, with $f_i \in \{-1, 0, 1\}$. We denote by $\text{idx}(f) := \{j : f_j \neq 0\}$ the set of indices corresponding to the non-zero coefficients of f .

For binary secret coefficients, the bootstrapping procedure only needs to distinguish whether a coefficient is zero or non-zero. In contrast, for ternary coefficients, the accumulator update also depends on the sign of each non-zero coefficient. Therefore, for every $j \in \text{idx}(f)$, we introduce an additional sign bootstrapping key $\mathbf{bsk}_j^{\text{sign}}$ that encrypts the sign bit $s_j := \frac{1-f_j}{2} \in \{0, 1\}$. Equivalently,

$$\mathbf{bsk}_j^{\text{sign}} := \begin{cases} \text{ACC.EncV}(1), & \text{if } f_j = -1, \\ \text{ACC.EncV}(0), & \text{if } f_j = 1. \end{cases} \quad (4)$$

During bootstrapping, the accumulator component ACC_k must be updated based on the signed contribution of f_j . Let $p := \text{ACC}_k \cdot X^{\bar{a}_k}$ and $z := \text{ACC}_k \cdot X^{-\bar{a}_k}$ denote the two possible signed rotations. Under our convention, p is selected when $f_j = 1$, and z is selected when $f_j = -1$. This selection can be implemented homomorphically as

$$(z - p) \boxplus \mathbf{bsk}_j^{\text{sign}} + p.$$

Indeed, if $f_j = 1$, $\mathbf{bsk}_j^{\text{sign}}$ encrypts 0, and the result is p , which corresponds to a rotation by $\bar{a}_k$; if $f_j = -1$, $\mathbf{bsk}_j^{\text{sign}}$ encrypts 1, and the result is z , which corresponds to a rotation by $-\bar{a}_k$. Thus, the same accumulator update can handle ternary coefficients.

Algorithm 2: Amortized Bootstrapping

Input: $\text{ct} \in \mathcal{R}_q$: base-scheme ciphertext encrypting a binary polynomial
 $m = \sum_{i=0}^{n-1} m_i X^i$;
 $\{\text{bsk}_{i,j}^{\text{rot}}\}_{0 \leq i < \ell_{\text{pos}}, 0 \leq j < h}$: $\text{bsk}_{i,j}^{\text{rot}}$ encrypts the i -th bit of $\mathbf{d}[j]$, where $\mathbf{d}$ is the position difference vector of $\text{idx}(f) = \{j_0, \dots, j_{h-1}\}$;
 $\{\text{bsk}_j^{\text{sign}}\}_{0 \leq j < h}$: sign-encoding keys for the h nonzero coefficients of f ;
 ksk^{aut} : key-switching key for the automorphism $X \mapsto X^{-1}$;
 ksk^{ext} : key-switching key from $f_Q \in \hat{\mathcal{R}}_Q$ to $f \in \mathcal{R}_q$
Output: $\text{ct}' \in \mathcal{R}_q$: a fresh base-scheme ciphertext encrypting the same m

- 1 $(\bar{a}_0, \dots, \bar{a}_{n-1}) \leftarrow \lfloor 2N \cdot \text{ct}/q \rfloor \in \mathbb{Z}_{2N}^n$;
- 2 $\text{ACC}_k \leftarrow \lfloor Q/8 \rfloor \cdot X^{N/2} \cdot \sum_{i=0}^{N-1} X^i \quad \text{for } 0 \leq k < n$;
- 3 **for** $0 \leq j < h$ **do**
- 4 **for** $0 \leq k < n$ **do**
- 5 $p \leftarrow \text{ACC}_k \cdot X^{\bar{a}_k}$;
- 6 $z \leftarrow \text{ACC}_k \cdot X^{-\bar{a}_k}$;
- 7 $\text{ACC}_k \leftarrow (z - p) \boxplus \text{bsk}_j^{\text{sign}} + p$; // sign rotation
- 8 $\text{ACC} \leftarrow \text{MPMul}(\text{ACC}, \text{bsk}_j^{\text{rot}}, \text{ksk}^{\text{aut}})$; // shift by $\mathbf{d}[j]$
- 9 $\text{ACC}_k \leftarrow \text{ACC}_k + \lfloor Q/8 \rfloor \cdot \sum_{i=0}^{N-1} X^i \quad \text{for } 0 \leq k < n$;
- 10 $\text{ACC} \leftarrow \{\text{ACC}_0, \dots, \text{ACC}_{n-1}\}$;
- 11 $\text{ACC} \leftarrow \text{ModSwitch}(\text{ACC})$; // $Q \rightarrow q$
- 12 $\text{ct}' \leftarrow \text{KeySwitch}(\text{ACC}, \text{ksk}^{\text{ext}})$; // $f_Q \rightarrow f$
- 13 **return** ct' ;

Amortized Bootstrapping Algorithm We now assemble the full amortized bootstrapping procedure based on `MPMul`. The key idea is to exploit the sparsity of the ternary secret key f and to evaluate the polynomial multiplication $a \cdot f$ as a sequence of monomial-by-polynomial multiplications.

Let $f = \sum_{j \in \text{idx}(f)} f_j X^j$ be a sparse ternary polynomial with Hamming weight h , where $f_j \in \{-1, 1\}$ for every $j \in \text{idx}(f)$. Let $n > j_0 > j_1 > \dots > j_{h-1} \geq 0$ be the non-zero indices of f , sorted in descending order. We define the difference vector $\mathbf{d} := \text{diff}(f)$ as

$$\mathbf{d}[i] = \begin{cases} j_i - j_{i+1}, & \text{if } 0 \leq i < h-1, \\ j_i, & \text{if } i = h-1. \end{cases}$$

By construction, $\mathbf{d}[i] \geq 0$ for all i . Here, R_{pos} is a key-generation parameter. When sampling the sparse ternary key, we check whether its difference vector satisfies $\mathbf{d}[i] < R_{\text{pos}}$ for all $0 \leq i < h$; otherwise, the key is rejected and a new sparse key is sampled. Hence, for every sampled key used in the scheme, we have $\mathbf{d} \in [0, R_{\text{pos}})^h$. This bounded-difference representation reduces the cost of each `MPMul` evaluation because the monomial exponent can be represented using only $\lceil \log_2 R_{\text{pos}} \rceil$ bits.

The multiplication $a \cdot f$ can be refactored into a nested sequence of rotations and additions. We define the recurrence $S_0 := a \cdot f_{j_0}$ and $S_{t+1} := \text{MPMul}(S_t, X^{\mathbf{d}[t]}) + a \cdot f_{j_{t+1}}$ for $0 \leq t < h-1$. By induction, the state after t steps is $S_t = \sum_{\ell=0}^t a \cdot f_{j_\ell} X^{j_\ell - j_t}$. Applying one final shift by $\mathbf{d}[h-1] = j_{h-1}$ reconstructs the full product: $\text{MPMul}(S_{h-1}, X^{\mathbf{d}[h-1]}) = \sum_{\ell=0}^{h-1} a \cdot f_{j_\ell} X^{j_\ell} = a \cdot f$.

In the homomorphic realization, each addition of $a \cdot f_{j_t}$ is carried out coefficient-wise by the sign-selection step above, each list shift $X^{\mathbf{d}[t]}$ is performed with one `MPMul` invocation, and each accumulator ACC_k is initialized with the test vector so that the refreshed message can be extracted from the resulting exponent. The procedure thus uses h `MPMul` invocations per packed ciphertext. The full procedure is presented in Algorithm 2.

4.2 Modulus Switching and Key Switching

The final stage of bootstrapping converts the accumulator output—which encodes the homomorphically decrypted message across n accumulator ciphertexts in the ACC.EncS form—back into the base encryption scheme, where the message is represented as a single polynomial ciphertext under NTRU.Enc . This stage consists of two steps: modulus switching, which reduces the ciphertext modulus from Q to q while keeping the ciphertext in $\hat{\mathcal{R}}$, followed by key switching, which transforms the resulting n ciphertexts in $\hat{\mathcal{R}}_q$ into a single ciphertext of the base scheme in $\mathcal{R}_q$.

Modulus Switching Let q and Q denote the moduli of the base and accumulator schemes, respectively. We perform modulus switching to map a ciphertext $c \in \hat{\mathcal{R}}_Q$ with coefficient vector $\mathbf{c} = (c_0, c_1, \dots, c_{N-1})^T$ to a ciphertext $c' \in \hat{\mathcal{R}}_q$ with coefficient vector $\mathbf{c}' = (c'_0, c'_1, \dots, c'_{N-1})^T$ by scaling each coefficient as $c'_i = \left\lfloor c_i \cdot \frac{q}{Q} \right\rfloor$, for $0 \leq i < N$.

Key Switching The construction incorporates two key switching procedures. The first transforms a ciphertext encrypted under $f_Q(X^{-1})$ into one encrypted under f_Q . This step is required by MPMu1 to handle the sign change of coefficients during rotation. The second step transforms the n ciphertexts of the accumulator scheme (under key f_Q) into a single ciphertext of the base scheme (under key f). We refer to the former as *automorphism key switching*, associated with key $\mathbf{ksk}^{\text{aut}}$, and to the latter as *external key switching*, using key $\mathbf{ksk}^{\text{ext}}$.

Automorphism Key Switching. Let $a \in \hat{\mathcal{R}}_Q$ be a ciphertext encrypting the i -th coefficient m_i of a message m under the secret key $s := f_Q(X^{-1})$, whose coefficient vector is $(f_{Q,0}, -f_{Q,N-1}, \dots, -f_{Q,1})^T$. The goal of automorphism key switching is to produce a new ciphertext $b \in \hat{\mathcal{R}}_Q$ that encrypts the same coefficient m_i under the secret key f_Q , whose coefficient vector is $(f_{Q,0}, f_{Q,1}, \dots, f_{Q,N-1})^T$. Following the structure of ACC.EncV , we define the automorphism key switching key as $\mathbf{ksk}^{\text{aut}} = \mathbf{h} \cdot f_Q^{-1} + \mathbf{g} \cdot s \in \hat{\mathcal{R}}_Q^{\ell_Q}$, where $\ell_Q = \lceil \log_B Q \rceil$ and $\mathbf{h} = (h_0, h_1, \dots, h_{\ell_Q-1})^T$ with each $h_i \leftarrow \chi_\zeta^N$. Let $(\tilde{a}_0, \dots, \tilde{a}_{\ell_Q-1}) := \mathbf{g}^{-1}(a)$ denote the gadget decomposition of a . The output ciphertext is then computed as

$$b = \sum_{i=0}^{\ell_Q-1} \mathbf{ksk}_i^{\text{aut}} \cdot \tilde{a}_i \in \hat{\mathcal{R}}_Q.$$

The correctness proof is deferred to Appendix D.

External Key Switching Let the output of modulus switching be $\text{ACC}' = \{b_k\}_{k=0}^{n-1}$, where each $b_k = \sum_{j=0}^{N-1} b_{k,j} X^j \in \hat{\mathcal{R}}_q$ is an accumulator ciphertext in ACC.EncS form. The constant coefficient of the phase $b_k f_Q$ contains the scaled encoding of the k -th target coefficient, together with its error. External key switching assembles these n accumulator ciphertexts into one ciphertext in the base-ring NTRU form under the key f .

Reorganization into N polynomials. Let $\text{ct}(\cdot)$ denote constant-coefficient extraction. Writing $\mathbf{f}_Q = (s_0, s_1, \dots, s_{N-1})^T$ and $\tilde{\mathbf{b}}_k := (b_{k,0}, -b_{k,N-1}, \dots, -b_{k,1})^T$, we have $\text{ct}(b_k f_Q) = \sum_{i=0}^{N-1} \tilde{b}_{k,i} s_i$. Therefore, the polynomial collecting the constant coefficients of the n accumulator phases can be written as

$$\Phi(X) := \sum_{k=0}^{n-1} \text{ct}(b_k f_Q) X^k = \sum_{k=0}^{n-1} \left(\sum_{i=0}^{N-1} \tilde{b}_{k,i} s_i \right) X^k = \sum_{i=0}^{N-1} s_i P_i(X), \quad (5)$$

where $P_i(X) := \sum_{k=0}^{n-1} \tilde{b}_{k,i} X^k \in \mathcal{R}_q$, $0 \leq i < N$. Under the correctness condition, $\Phi(X)$ is the scaled encoding of the target message polynomial plus the accumulated error. This

Table 2: Summary of auxiliary keys used in amortized bootstrapping.

Name	Purpose / Operation
BSK^{rot}	Rotation for sparse-position differences
BSK^{sign}	Sign selection for ternary coefficients
KSK^{aut}	Automorphism key switching ($X \mapsto X^{-1}$)
KSK^{ext}	Accumulator-to-base key switching

regrouping converts the n length- N phase computations into N polynomial operations in the base ring, with each source-key coefficient s_i appearing as a scalar.

Key generation and switching. Following the structure of $\text{ACC}.\text{EncV}$ but adapted to the base ring $\mathcal{R}_q$, we define one gadget-decomposed key per index $i \in [0, N)$ that encrypts the source-key (accumulator scheme) coefficient s_i under the target key (base scheme) f : $\text{ksk}_i^{\text{ext}} = \mathbf{h}_i \cdot f^{-1} + \mathbf{g} \cdot s_i \in \mathcal{R}_q^{\ell_q}$, for $0 \leq i < N$, where $\mathbf{h}_i = (h_{i,0}, \dots, h_{i,\ell_q-1})^T$ with each $h_{i,j} \leftarrow \chi_\sigma^n$, and $\ell_q = \lceil \log_B q \rceil$. The total key consists of $N \cdot \ell_q$ polynomials in $\mathcal{R}_q$.

The target ciphertext in base NTRU form is then computed by gadget-decomposing each P_i and accumulating the inner products with the corresponding key:

$$c(X) = \sum_{i=0}^{N-1} \langle \mathbf{g}^{-1}(P_i), \text{ksk}_i^{\text{ext}} \rangle \in \mathcal{R}_q.$$

The correctness proof is deferred to Appendix E. We summarize the auxiliary keys used in amortized bootstrapping in Table 2.

4.3 Complexity and Key-Size Analysis

We compare the core bootstrapping computation of our amortized construction with the FINAL [BIP⁺22], which serves as the closest NTRU-based baseline. Throughout this comparison, $\ell_Q := \lceil \log_B Q \rceil$ denotes the accumulator gadget level, h denotes the Hamming weight of the sparse NTRU secret, and $\ell_{\text{pos}} := \lceil \log_2 R_{\text{pos}} \rceil$ denotes the number of bits used to encode a bounded sparse-position difference.

A ciphertext in our base ring packs n message coefficients. The amortized procedure uses h MPMul invocations. For each nonzero secret coefficient, it performs one sign-selection external product per output and ℓ_{pos} conditional-rotation external products per output. The conditional-selection computation therefore uses $\mathcal{O}(nh(\ell_{\text{pos}} + 1)) = \mathcal{O}(nh\ell_{\text{pos}})$ external products per packed ciphertext, or $\mathcal{O}(h(\ell_{\text{pos}} + 1)) = \mathcal{O}(h\ell_{\text{pos}})$ per output coefficient. The rotation bootstrapping key contains $h\ell_{\text{pos}}$ EncV ciphertexts, and the sign bootstrapping key contains h additional EncV ciphertexts. Since each EncV ciphertext contains ℓ_Q accumulator-ring elements, the combined size is $h(\ell_{\text{pos}} + 1)\ell_Q$ accumulator-ring elements. In contrast, the bootstrapping key size of the base version has size $n\ell_Q$.

FINAL uses a matrix-NTRU representation in which each ciphertext encodes a single message bit. Its bootstrapping procedure evaluates a single decryption inner product of length n , requiring $\mathcal{O}(n)$ accumulator-external products, or $\mathcal{O}(n\ell_Q)$ accumulator-ring multiplications, per output bit. Processing n independent output bits therefore requires $\mathcal{O}(n^2)$ accumulator-external products. The corresponding bootstrapping key contains $\mathcal{O}(n\ell_Q)$ accumulator-ring elements. Table 3 compares the core external-product computation with the associated bootstrapping keys.

Table 3: Complexity comparison.

Method	Per output	n outputs	BSK size
FINAL [BIP ⁺ 22]	$\mathcal{O}(n\ell_Q)$	$\mathcal{O}(n^2\ell_Q)$	$n\ell_Q$
Proposed	$\mathcal{O}(h\ell_{\text{pos}})$	$\mathcal{O}(nh\ell_{\text{pos}})$	$h(\ell_{\text{pos}} + 1)\ell_Q$

5 Security and Noise Analysis

5.1 Security Model and Assumptions

IND-CPA Security The security of both encryption schemes and all auxiliary keys reduces to the decisional NTRU assumption over their respective rings $\mathcal{R}_q$ and $\hat{\mathcal{R}}_Q$. Under this assumption, fresh ciphertexts—including the base scheme $c = (g + \Delta \cdot m) \cdot f^{-1}$ and the accumulator forms `ACC.EncS` and `ACC.EncV`—are computationally indistinguishable from uniform samples due to randomization with fresh NTRU-distributed noise terms. Similarly, the pseudorandomness of all auxiliary keys is guaranteed by the same encryption scheme, under the standard circular-security assumption required for bootstrapping.

Circular Security Bootstrapping additionally publishes encryptions of key material under related keys (e.g., BSK^{rot} and ksk^{ext}), as in FHEW [DM15], TFHE [CGGI16], and FINAL [BIP⁺22], we assume weak circular security: these key-dependent encryptions remain indistinguishable from encryptions of independent messages. No known attacks exploit this circular structure in NTRU-based schemes.

5.2 Avoiding overstretched NTRU

For concrete security, we bound the admissible ciphertext moduli using the *NTRU Fatigue* script by Ducas and van Woerden [DvW21], which verifies parameters against sublattice attacks. We set the key-distribution variance to $\sigma^2 = 0.5$, corresponding to our ternary key distribution with $\Pr[X = 0] = \frac{1}{2}$ and $\Pr[X = 1] = \Pr[X = -1] = \frac{1}{4}$. The NTRU type is set to *circulant* in the script to distinguish it from the matrix variant; we argue that the anti-circulant ring used in our scheme does not weaken security relative to the circulant case considered in the script.

To convert a BKZ block size β into a security level, we adopt the same classical cost model used by TFHE [CGGI16] and FINAL [BIP⁺22]: a security level of λ bits requires $T(d, \beta) \geq 2^\lambda$, where $T(d, \beta) := 2^{0.292 \cdot \beta + 16.4 + \log_2(8 \cdot d)}$, and $d = 2n$ for an NTRU ciphertext in dimension n . We write β_{128} for the smallest block size such that $T(d, \beta_{128}) \geq 2^{128}$ at the respective lattice dimension.

Rather than fixing the moduli in advance, we use the script to determine, for each candidate ring dimension, the largest modulus that remains outside the overstretched regime and still yields a predicted attack block size that meets β_{128} . Table 4 reports these boundaries, quantized to the largest NTT-friendly primes supported by our implementation, along with β_{128} and the corresponding certified level $\lambda = \log_2 T(d, \beta_{128})$. The boundary at dimension n upper-bounds the base modulus q , and the boundary at dimension N upper-bounds the accumulator modulus Q ; the parameter selection in Section 6.2 draws moduli only from within these limits.

Since every admissible modulus lies outside the overstretched regime, sublattice attacks are no easier than generic key-recovery attacks on lattice-based schemes, and the BKZ-based bound above provides a meaningful estimate of the classical security level. The Hamming weight h of the sparse secret introduces an additional attack surface, which we evaluate separately in the next subsection using the Lattice Estimator [DvW21].

Table 4: Largest moduli attaining 128-bit NTRU security.

Dimension	Maximal modulus		β_{128}	λ
1024	925,697	$(\approx 2^{19.82})$	335	128.22
2048	1,208,020,993	$(\approx 2^{30.17})$	331	128.05
4096	48,734,206,795,777	$(\approx 2^{45.47})$	328	128.18
8192	266,002,186,081,541,160,961	$(\approx 2^{67.85})$	324	128.01

5.3 Sparse-Key Security

In amortized bootstrapping, we use a secret key with Hamming weight h , which is known to be vulnerable to the dual hybrid attack with meet-in-the-middle optimization [CHHS19]. We therefore evaluate every candidate key configuration with the Lattice Estimator [DvW21]: for each candidate Hamming weight h and each candidate base modulus q below the corresponding boundary in Table 4, we denote by $\lambda_0(h, q)$ the resulting security level, defined as the minimum **rop** value across all attacks supported by the estimator. Throughout this analysis, the noise distribution is fixed at $\sigma = \sqrt{1/2} \approx 0.707$.

The per-message cost of amortized bootstrapping scales with both h and $\ell_{\text{pos}} = \lceil \log_2 R_{\text{pos}} \rceil$, so small values of both are preferable. However, the rejection sampling used to enforce the position-bound R_{pos} conditions the key distribution, thereby compromising security. We therefore define the effective security of a candidate (h, q, R_{pos}) as $\lambda := \lambda_0(h, q) - \delta$, where δ is the rejection-induced loss quantified below. We deem a candidate admissible only if $\lambda \geq 128$. The joint selection of (h, R_{pos}) and the remaining scheme parameters over this admissible space is deferred to the runtime-driven optimization in Section 6.2.

Rejection Sampling To reduce the per-step cost of the external product, we apply rejection sampling during key generation and compute the corresponding security loss following the methodology of Guimarães and Pereira [GP25]. Let $d := \text{diff}(f)$ denote the sequence of gaps between consecutive non-zero positions of the sparse secret f , and let R_{pos} be an upper bound on these gaps. During key generation, we accept a candidate key only if $|d[i]| < R_{\text{pos}}$, for all $0 \leq i < h$.

To quantify the impact of this restriction on the key distribution, we sample a large number of random keys (e.g., 10^5), compute $M(f) := \max_{0 \leq i < h} |d[i]|$, and estimate the acceptance probability as $p_{\text{acc}}(R_{\text{pos}}) := \Pr[M(f) < R_{\text{pos}}]$. The security loss induced by rejection sampling is then defined as $\delta := -\log_2 p_{\text{acc}}(R_{\text{pos}})$.

Equivalently, the rejection-conditioned key distribution loses approximately δ bits of security compared with uniform sampling over the original sparse-key support. We discard candidate (h, R_{pos}) pairs with low acceptance probability because NTRU key generation independently rejects non-invertible candidates and a low acceptance rate would compound the two rejection conditions, making sampling impractical.

Table 5 summarizes, for each ring dimension n , the sparse-key configuration selected by the optimization: the estimator output λ_0 (at the selected base modulus q), the acceptance probability p_{acc} together with the induced loss δ , and the effective security $\lambda = \lambda_0 - \delta$, which exceeds the 128-bit target in all cases.

5.4 Noise Analysis and Correctness

Noise Model and Notation We track noise by the *variance* of each ciphertext’s error term. Assuming all noise sources are independent and subGaussian, we convert variance into a high-probability magnitude bound using the central-limit heuristic. This yields an average-case analysis in the spirit of FHEW [DM15], TFHE [CGGI16], and FINAL [BIP+22]. For

Table 5: Sparse-key security analysis.

Dimension n	Hamming weight h	R_{pos}	λ_0	p_{acc}	δ	λ
1024	78	2^6	128.80	0.62	0.70	128.10
2048	46	2^7	132.90	0.036	4.80	128.10
4096	32	2^9	129.20	0.61	0.72	128.48
8192	26	2^{10}	129.90	0.36	1.46	128.44

a vector ciphertext $\mathbf{c}$, we write $E(\mathbf{c}) := \max_j \text{Var}(\text{err}(\mathbf{c}_j))$, and for a scalar ciphertext c we use $E(c) := \text{Var}(\text{err}(c))$. The notation used in this section is summarized below.

- σ, ς : the noise standard deviations for the base and accumulator schemes, respectively.
- B_q, B_Q, B_{aut} : gadget bases for the base-scheme, accumulator, and automorphism key-switching gadgets, respectively.
- $\ell_q := \lceil \log_{B_q} q \rceil$, $\ell_Q := \lceil \log_{B_Q} Q \rceil$, and $\ell_{\text{aut}} := \lceil \log_{B_{\text{aut}}} Q \rceil$: decomposition levels for the base, accumulator, and automorphism key-switching keys, respectively.
- γ^2 : variance of a single gadget digit, assuming $\mathbf{g}^{-1}(\cdot)$ is γ -subGaussian. Since the gadget base B is even, the digits lie in the *asymmetric* interval $[-B/2, B/2)$ and have mean $-1/2$ and variance $\gamma^2 = (B^2 + 2)/12$.
- $\ell_{\text{pos}} := \lceil \log_2 R_{\text{pos}} \rceil$: rotation-decomposition depth in **MPMu1**, where R_{pos} is the position bound in Section 4.1.

Fresh Ciphertexts, NAND and External Product A fresh base-scheme ciphertext $c = (g + \Delta \cdot m) \cdot f^{-1}$ has phase $c \cdot f = g + \Delta \cdot m$, so its noise is $g \leftarrow \chi_\sigma^n$. The NAND gate output $c_{\text{NAND}} = \mathbf{evk} - c_0 - c_1$ accumulates three independent fresh-noise terms, along with the rounding error from scaling. By Lemma 2, correctness of NAND requires $\|\mathbf{g}_0 + \mathbf{g}_1\|_\infty < \frac{q-20}{8}$, which translates to a per-coefficient threshold of $q/16 - 5/4$.

The external product is the fundamental building block of all bootstrapping operations. We present a single bound and reuse it below.

Lemma 3 (External-product noise). *Let c be a scalar ciphertext encrypting u , and let $\mathbf{c}$ a vector ciphertext encrypting $v \in \mathbb{M}$ in the accumulator scheme. Then the noise variance of $c_{\text{out}} := c \boxtimes \mathbf{c}$ satisfies*

$$E(c_{\text{out}}) \leq E(c) + N \cdot \ell_Q \cdot \gamma_Q^2 \cdot E(\mathbf{c}).$$

Amortized Bootstrapping Amortized bootstrapping consists of an initial test vector, h iterations—each consisting of one **MPMu1** call and one sign-correction external product—modulus switching from Q to q , and external key switching back to the base scheme. The initial accumulator and the two switching procedures contribute:

- Initial accumulator noise: $E_{\text{Acc}_0} \leq 4N\varsigma^2$.
- Modulus switching: $E_{\text{ms}} := 1 + 16N\varsigma^2$.
- External key switching: $E_{\text{ks}} := n \cdot N \cdot \ell_q \cdot \gamma_q^2 \cdot E(\mathbf{ksk}^{\text{ext}})$.

For the per-iteration terms, let

$$\begin{aligned} \Delta E_{\text{ep}} &:= N \cdot \ell_Q \cdot \gamma_Q^2 \cdot E(\mathbf{bsk}^{\text{rot}}), \\ E_{\text{ks}^{\text{aut}}} &:= N \cdot \ell_{\text{aut}} \cdot \gamma_{\text{aut}}^2 \cdot E(\mathbf{ksk}^{\text{aut}}), \\ E_{\text{sign}} &:= N \cdot \ell_Q \cdot \gamma_Q^2 \cdot E(\mathbf{bsk}^{\text{sign}}). \end{aligned}$$

A single MPMu1 call inflates the noise by at most $E_{\text{MPMu1}} \leq \ell_{\text{pos}} \cdot (\Delta E_{\text{ep}} + E_{\text{ks}^{\text{aut}}})$. After h such iterations, together with modulus and external key switching, the output noise satisfies

$$E_{\text{out}} \leq \left(\frac{q}{Q}\right)^2 (E_{\text{acc}_0} + h \cdot (E_{\text{MPMu1}} + E_{\text{sign}})) + E_{\text{ms}} + E_{\text{ks}}.$$

Correctness Under the central-limit heuristic, each error coefficient in a refreshed ciphertext approximately follows a centered Gaussian distribution with variance E_{out} . We measure correctness by the *per-slot* failure probability: $p_{\text{fail}} := \Pr\left[|e| \geq \frac{q}{16} - \frac{5}{4}\right]$, $e \sim \mathcal{N}(0, E_{\text{out}})$.

Theorem 1 (Bootstrapping correctness, average case). *For the parameter sets selected in Section 6.2, each error coefficient e of the refreshed ciphertext c' produced by amortized bootstrapping satisfies $|e| < \frac{q}{16} - \frac{5}{4}$, except with probability p_{fail} over the randomness of the fresh noise samples, and hence $\text{NTRU.Dec}(c')$ recovers each refreshed message with probability at least $1 - p_{\text{fail}}$.*

6 Implementation and Evaluation

6.1 Implementation

We implemented the proposed polynomial-NTRU FHE scheme with sparse-key amortized bootstrapping in C++. Polynomial multiplication dominates both external products and key switching, so we accelerate these operations using Intel HEXL [BKS⁺21] for NTT-based multiplication and coefficient-wise modular arithmetic. As required by HEXL, we restrict the ring dimensions to powers of two and choose moduli such that $q \equiv 1 \pmod{2n}$ and $Q \equiv 1 \pmod{2N}$; this NTT-friendliness constraint is already reflected in the modulus boundaries of Table 4.

Approximate Gadget Decomposition An external product with decomposition length ℓ requires $\ell + 1$ transforms— ℓ forward NTTs for the digit polynomials plus one inverse NTT—so ℓ directly scales the dominant cost of bootstrapping. To shorten the decomposition beyond what an exact gadget allows, we also support an *approximate* signed gadget: for a power-of-two base $B = 2^b$ and modulus M , the least significant **drop** bits of the input are rounded away before decomposition, leaving $\ell = \lceil (\lceil \log_2 M \rceil - \text{drop})/b \rceil$ digits instead of $\lceil \log_B M \rceil$. For instance, reducing ℓ_Q from 2 to 1 removes one-third of the transforms for each external product.

The rounding introduces an error ε uniformly distributed on $(-2^{\text{drop}-1}, 2^{\text{drop}-1}]$, so $\text{Var}(\varepsilon) = 4^{\text{drop}}/12$. In an external product, this error is multiplied by the scaled accumulator secret $f_Q = 1 + 4f'_Q$, whose coefficients have variance $\sigma_{f_Q}^2 = 16 \cdot \frac{1}{2} = 8$ under our ternary key distribution. Thus, every invocation of an approximate gadget adds $\Delta E_{\text{drop}} := N \cdot \frac{4^{\text{drop}}}{12} \cdot \sigma_{f_Q}^2$ to the corresponding per-invocation increment in Section 5.4: ΔE_{ep} and E_{sign} gain the term with drop_Q , $E_{\text{ks}^{\text{aut}}}$ with drop_{aut} , and the external key switching gains ΔE_{drop} with drop_q —in that case without the factor n , since the rounding couples only to the N scalar coefficients. An exact decomposition ($\text{drop} = 0$) incurs no such term. Whether a shorter-but-noisier gadget pays off therefore depends on the remaining noise budget, which is determined by the parameter search below.

6.2 Parameter Selection

For each base dimension n , we select all remaining parameters—the accumulator dimension N , the moduli q and Q , the sparse-key pair (h, ℓ_{pos}) , and the three gadget configurations $(B_q, \ell_q, \text{drop}_q)$, $(B_Q, \ell_Q, \text{drop}_Q)$, and $(B_{\text{aut}}, \ell_{\text{aut}}, \text{drop}_{\text{aut}})$ —via a joint search that minimizes

Table 6: Parameter sets selected by the joint optimization.

n	N	q	Q	h	R_{pos}	$(B_q, \ell_q, \text{drop}_q)$	$(B_Q, \ell_Q, \text{drop}_Q)$	$(B_{\text{aut}}, \ell_{\text{aut}}, \text{drop}_{\text{aut}})$
1024	1024	796,673	925,697	78	2^6	(2, 15, 5)	(16, 5, 0)	(4, 10, 0)
2048	1024	946,177	925,697	46	2^7	(4, 7, 6)	(16, 5, 0)	(8, 7, 0)
4096	1024	1,892,353	925,697	32	2^9	(4, 8, 5)	(16, 5, 0)	(16, 5, 0)
8192	1024	5,767,169	925,697	26	2^{10}	(16, 4, 7)	(16, 5, 0)	(16, 5, 0)

the predicted bootstrapping runtime over the space defined by the security analysis in Sections 5.2 and 5.3 and the correctness requirement in Section 5.4:

- *Moduli.* q and Q range over NTT-friendly primes below the bounds in Table 4 for dimensions n and N , respectively, with $N \in \{1024, 2048, 4096, 8192\}$.
- *Sparse key.* A pair (h, ℓ_{pos}) is admissible only if $\lambda_0(h, q) - \delta(h, \ell_{\text{pos}}) \geq 128$, where λ_0 is precomputed once using the Lattice Estimator on a per-dimension (h, q) grid, and δ is estimated via Monte-Carlo sampling of the gap distribution.
- *Correctness.* The model-predicted per-slot failure probability—computed from the closed-form bound E_{out} in Section 5.4 and extended by the approximate-gadget terms ΔE_{drop} in Section 6.1—must satisfy $p_{\text{fail}} \leq 2^{-64}$.

Since transforms dominate the runtime, the objective is a transform-weighted proxy: each k -dimensional NTT is weighted by its cost $C_k := k \log_2 k$, enabling direct comparison of configurations with different accumulator dimensions. Counting the forward and inverse transforms of all external products, the automorphism key switching, and the final external key switching yields

$$\text{Cost} = \left(h n [(\ell_{\text{pos}} + 1)(\ell_Q + 1) + 1] + h (2^{\ell_{\text{pos}}} - 1)(\ell_{\text{aut}} + 1) \right) C_N + (N \ell_q + 1) C_n.$$

The search proceeds in two levels. The outer level iterates over the admissible space $(N, q, Q, h, \ell_{\text{pos}})$. For each such tuple, the correctness target is converted to a noise budget $E_{\text{out}} \leq E_{\text{max}}(q)$ using the complementary error function, and the inner level allocates this budget across the three gadgets: for each gadget and each achievable length ℓ we precompute the noise-minimal pair (B, drop) —a Pareto frontier over bases and drop amounts—and then scan the combinations of lengths, keeping the cheapest one whose total noise fits the budget. Approximate gadgets are thus selected automatically whenever the transforms they save outweigh the noise they introduce. Table 6 reports the resulting parameter sets; by construction, each set provides at least 128-bit classical security and a per-slot failure probability below 2^{-64} . Notably, the search converges to a single accumulator configuration $(N, Q) = (1024, 925,697)$ —the boundary modulus of Table 4—for all four base dimensions, and uses the approximate gadget only on the base-ring side ($\text{drop}_Q = \text{drop}_{\text{aut}} = 0$). Discarding 5–7 low-weight bits shortens ℓ_q to as little as 4 and, as a side effect, shrinks the external key-switching key.

6.3 Experimental Setup

All experiments were conducted on an AWS EC2 **c8a.xlarge** instance running Ubuntu 24.04 LTS. The proposed framework is implemented in C++ with Intel HEXL acceleration, as described in Section 6.1. For performance comparisons, we benchmarked the reference implementation of FINAL [BIP⁺22], the leading NTRU-based FHE scheme; the highly optimized Rust implementation of TFHE [CGGI16, Zam22] (Zama **tfhe-rs**) in its **Boolean** configuration, the state of the art in single-bit LWE-based bootstrapping; and the amortized bootstrapping implementation of Guimarães and Pereira (GP25) [GP25], which provides

Table 7: Bootstrapping performance comparison. Represented in milliseconds (ms).

Scheme	n	Latency	Amortized	vs FINAL	vs TFHE	vs GP25
FINAL	1	121.20	121.20	—	—	—
TFHE	1	7.10	7.10	—	—	—
GP25	2048	5184.67	2.53	—	—	—
GP25	4096	9662.89	2.36	—	—	—
GP25	8192	18737.01	2.29	—	—	—
Proposed	1024	5143.73	5.02	24.14×	1.41×	—
Proposed	2048	6990.68	3.41	35.54×	2.08×	0.74×
Proposed	4096	12314.39	3.01	40.27×	2.36×	0.78×
Proposed	8192	21964.20	2.68	45.22×	2.65×	0.85×

parameter sets only for $n \in \{2048, 4096, 8192\}$ — all on the same hardware configuration. Unless otherwise specified, all evaluations were performed in a single-threaded setting without GPU acceleration to ensure a fair and consistent architectural comparison.

To keep the correctness targets aligned, GP25 is configured with the fastest available key-switching parameter t_{ks} whose failure probability still meets our target $p_{fail} \leq 2^{-64}$. Due to the coarse granularity of t_{ks} , the resulting configurations fall well below the target (between 2^{-101} and 2^{-118}), but the next faster setting would already violate it. TFHE’s default parameters target 2^{-64} , matching ours, while FINAL’s reference parameters correspond to $p_{fail} \leq 2^{-59.7}$, slightly above the target; we nevertheless report its stock configuration.

6.4 Performance

Table 7 reports the bootstrapping performance of our scheme against these three baselines. Each entry reports average wall-clock times measured over 100 runs: per-batch latency and amortized time per bootstrapped *message coefficient*. For amortized schemes, the latter equals the per-batch time divided by the number of refreshed messages; for FINAL and TFHE, which encrypt a single bit per ciphertext, the two coincide.

The per-coefficient cost of the proposed scheme decreases monotonically with the packing dimension, from 5.02 ms at $n = 1024$ to 2.68 ms at $n = 8192$, consistent with the $\mathcal{O}(h\ell_{pos})$ per-coefficient complexity established in Section 4.3. The admissible Hamming weight h shrinks as n grows, so amortization improves with the packing factor. A single accumulator dimension $N = 1024$ suffices across all parameter sets.

On a per-coefficient basis, the proposed scheme outperforms FINAL by 24.14× to 45.22× and the highly optimized TFHE baseline (7.10 ms per bit) by 1.41× to 2.65×, while each packed ciphertext additionally carries n message bits. Compared with GP25, our per-coefficient cost is 1.17× to 1.35× higher, with the gap narrowing as n grows. We note that GP25 also relies on a two-key accumulator technique that our implementation does not incorporate; whether and to what extent this technique applies to our polynomial-NTRU setting is left for future work.

6.5 Key and Ciphertext Size

We measure storage overhead by direct measuring the bootstrapping keys and ciphertexts in memory. Unlike theoretical estimates, these figures reflect the actual memory footprint of live objects at runtime. Table 8 breaks the auxiliary keys into the bootstrapping key (BSK) and the external key-switching key (KSK).

The BSK highlights the advantage of the amortized approach: it does not scale with the packing dimension n but instead depends on the Hamming weight h and the position bound R_{pos} . It shrinks from 21.53 MB at $n = 1024$ to 11.28 MB at $n = 8192$ —4× to

Table 8: Ciphertext and auxiliary key sizes.

Scheme	n	Ctxt (KB)	B/bit	BSK (MB)	KSK (MB)	Total (MB)
FINAL	1	3.15	3224.0	86.41	34.89	121.30
TFHE	1	3.18	3260.0	—	—	124.43
GP25	2048	32.06	16.03	18.02	128.58	146.60
GP25	4096	64.06	16.02	16.89	256.86	273.75
GP25	8192	128.06	16.01	16.51	513.42	529.93
Proposed	1024	8.07	8.07	21.53	121.08	142.61
Proposed	2048	16.07	8.04	14.51	112.52	127.03
Proposed	4096	32.07	8.02	12.62	256.59	269.21
Proposed	8192	64.07	8.01	11.28	256.30	267.58

Table 9: Failure probabilities.

n	1024	2048	4096	8192
Predicted	$2^{-64.0}$	$2^{-64.2}$	$2^{-64.4}$	$2^{-64.5}$
Measured	$2^{-69.8}$	$2^{-76.8}$	$2^{-110.5}$	$2^{-103.7}$

$7.7\times$ smaller than FINAL’s bootstrapping key and consistently below GP25’s. The total footprint is instead dominated by the KSK, which scales with $n\ell_q$; here the approximate base-ring gadget of Section 6.1 pays off a second time because discarding low-weight bits shortens ℓ_q and, consequently, the KSK. Overall, the proposed scheme requires less key material than GP25 across all common dimensions ($1.15\times$ less at $n = 2048$, and up to $1.98\times$ at $n = 8192$).

Regarding ciphertext size, our individual packed ciphertexts are larger than the single-bit ciphertexts of FINAL and TFHE, but they are significantly more compact per message: normalized by the packing factor n , the storage cost is about 8 bytes per bit across all dimensions—over $400\times$ lower than the single-bit constructions and half of GP25’s ~ 16 bytes per slot. This combination of compact bootstrapping keys and high per-slot density makes the proposed scheme particularly attractive when ciphertext bandwidth is at a premium.

6.6 Failure Probability

To document the correctness margin of the selected parameter sets, we evaluate the per-slot failure probability in Section 5.4,

$$p_{\text{fail}} \approx \operatorname{erfc}\left(\frac{q/16 - 5/4}{\sigma_{\text{out}}\sqrt{2}}\right),$$

in two ways: with the model prediction $\sigma_{\text{out}} = \sqrt{E_{\text{out}}}$ used by the parameter search (predicted) and with the empirical standard deviation of the observed output noise (measured). The latter is computed over 100 bootstrapping trials per parameter set: each trial encrypts two fresh random binary vectors, evaluates a coefficient-wise NAND gate, bootstraps the result, and records the signed error for each output coefficient against the ground truth, yielding $100n$ error samples per set; no decryption failures occurred in any run. Table 9 lists both. The predicted values fall directly on the target ($2^{-64.0}$ to $2^{-64.5}$) across all dimensions, indicating that the joint optimization in Section 6.2 uses the full noise budget to achieve the target performance. The measured noise is 4%–24% below the bound, yielding failure probabilities between $2^{-69.8}$ and $2^{-110.5}$, which confirms that the average-case analysis is conservative yet reasonably tight.

7 Conclusion and Future Work

We revisited NTRU-based FHE, motivated by the refined security analysis of Ducas and van Woerden [DvW21] and the FINAL construction of Bonte et al. [BIP⁺22]. Reformulating FINAL’s matrix-based construction in the standard polynomial-ring setting restores the algebraic structure required for automorphism-based amortization; combining an NTRU-compatible MPMul with sparse ternary secrets then reduces the per-coefficient bootstrapping cost from $\mathcal{O}(n\ell_Q)$ to $\mathcal{O}(h\ell_{\text{pos}})$ and corresponding bootstrapping-key size. We further shortens the gadget length through an approximate base-ring decomposition. Concrete security is established by a two-step parameter selection procedure that combines the NTRU fatigue analysis with the Lattice Estimator under a given failure probability target.

Our single-threaded C++/HEXL implementation refreshes a message coefficient in 2.68 ms at $n = 8192$ —up to $45\times$ faster than FINAL and $2.65\times$ faster than TFHE-rs—with an 11.28 MB bootstrapping key that does not grow with n and packed ciphertexts of about 8 bytes per message bit. These results establish polynomial-NTRU FHE as a competitive alternative to mainstream bitwise schemes and the more economical option in key- and bandwidth-constrained deployments.

The most immediate open question is whether GP25’s two-key accumulator technique applies to the polynomial-NTRU setting, which would close the remaining $1.17\times$ to $1.35\times$ runtime gap. Beyond this, compressing the key-switching key—now the dominant part of the total footprint—and exploiting multi-threading or GPU acceleration are natural next steps.

A Proof of Lemma 1

Proof. Since $c \cdot f = g + \Delta \cdot m \pmod{q}$, the phase equals $r := c \cdot f = g + \Delta \cdot m - uq$ for some $u \in \mathbb{Z}$. Writing $\Delta = \lfloor q/2 \rfloor = q/2 + \epsilon$ with $|\epsilon| \leq 1/2$, decryption computes

$$m' = \left\lfloor \frac{2r}{q} \right\rfloor = \left\lfloor \frac{2g + 2\epsilon m}{q} + m - 2u \right\rfloor = \left\lfloor \frac{2g + 2\epsilon m}{q} \right\rfloor + m \pmod{2},$$

where the integer term $m - 2u$ is pulled outside the rounding and $-2u$ vanishes modulo 2. Hence $m' = m$ whenever $|(2g_i + 2\epsilon m_i)/q| < 1/2$ for all $0 \leq i < n$. Since g is ternary, m binary, and $|\epsilon| \leq 1/2$, the term is bounded by $3/q < 1/2$ for $q > 6$, hence $m' = m$.

B Proof of Lemma 2

Proof. Write $\lfloor q/4 \rfloor = q/4 + \epsilon'$ and $\lfloor 5q/8 \rfloor = 5q/8 + \epsilon$ with $|\epsilon|, |\epsilon'| \leq 1/2$. Using the identity $(a - b)^{\circ 2} = a + b - 2(a \circ b)$ from Section 2.1,

$$\begin{aligned} c_{\text{NAND}} \cdot f &= (\text{evk} - c_0 - c_1) \cdot f \\ &= \left(g + \frac{5q}{8} + \epsilon\right) - \left(g_0 + \frac{q}{4}a + \epsilon'a\right) - \left(g_1 + \frac{q}{4}b + \epsilon'b\right) \\ &= g - g_0 - g_1 + e + \delta + \frac{q}{2}y, \end{aligned}$$

where $y := 1 - a \circ b$, $e := \epsilon - \epsilon'(a + b)$ satisfies $|e_i| \leq 1/2 + (1/2) \cdot 2 = 3/2$, and $\delta := \frac{q}{8} - \frac{q}{4}(a - b)^{\circ 2}$ has entries $\delta_i = \pm q/8$. As in Lemma 1, since $y \in \{0, 1\}^n$ can be pulled outside the rounding, decryption gives

$$y' = \left\lfloor \frac{2(g - g_0 - g_1 + e) + 2\delta}{q} \right\rfloor + y \pmod{2},$$

so $y'_i = y_i$ whenever $|2(g_i - g_{0,i} - g_{1,i} + e_i)/q \pm 1/4| < 1/2$, which holds in particular when $|g_i - g_{0,i} - g_{1,i} + e_i| < q/8$. By the triangle inequality with $|g_i| \leq 1$ and $|e_i| \leq 3/2$, this is guaranteed by the lemma's hypothesis

$$|g_{0,i} + g_{1,i}| < \frac{q}{8} - \frac{5}{2} = \frac{q-20}{8}, \quad \text{for all } 0 \leq i < n.$$

Therefore $y' = y$, completing the proof. $\square$

C Correctness of the External Product

Recall $c = g/f_Q + \Delta \cdot u$ and $\mathbf{c} = \mathbf{g}/f_Q + \mathbf{g} \cdot v$, where $\mathbf{g} = (g_0, \dots, g_{\ell_Q-1})$ collects fresh noise polynomials and $v \in \mathbb{M}$ is a signed monomial. By the defining property $\mathbf{g}^{-1}(c) \cdot \mathbf{g} = c$ of the gadget decomposition,

$$c \boxdot \mathbf{c} = \mathbf{g}^{-1}(c) \cdot \mathbf{c} = \mathbf{g}^{-1}(c) \cdot \mathbf{g}/f_Q + c \cdot v = \underbrace{(\mathbf{g}^{-1}(c) \cdot \mathbf{g} + g \cdot v)}_{=: g_{\text{mult}}}/f_Q + \Delta \cdot u \cdot v,$$

which has exactly the form of a scalar encryption of $u \cdot v$ in the accumulator scheme. Correctness requires g_{mult} to stay within the rounding tolerance and $\|u \cdot v\|_\infty < 4$; the latter is immediate since v is a signed monomial and u is ternary.

D Proof of Automorphism Key Switching

Proof. Let $\tilde{s} := f_Q(X^{-1})$ and let $\tilde{c} \in \hat{\mathcal{R}}_Q$ satisfy $\tilde{c} \cdot \tilde{s} = \tilde{g} + \Delta \tilde{u}$. With $c_i := \mathbf{g}^{-1}(\tilde{c})_i$, so that $\sum_i c_i \cdot B^i = \tilde{c}$, and $\mathbf{ksk}_i^{\text{aut}} = (h_i + \tilde{s} \cdot B^i) \cdot f_Q^{-1}$ with fresh noise $h_i \leftarrow \chi_\zeta^N$, the output $c' = \sum_{i=0}^{\ell_Q-1} \mathbf{ksk}_i^{\text{aut}} \cdot c_i$ satisfies

$$c' \cdot f_Q = \sum_{i=0}^{\ell_Q-1} (h_i + \tilde{s} \cdot B^i) \cdot c_i = \sum_{i=0}^{\ell_Q-1} h_i \cdot c_i + \tilde{s} \cdot \tilde{c} = \underbrace{\sum_{i=0}^{\ell_Q-1} h_i \cdot c_i}_{=: g'} + \Delta \tilde{u}.$$

Hence c' is a valid encryption of $\tilde{u}$ under the target key f_Q , and decrypts correctly provided g' remains within the decryption tolerance. $\square$

E Proof of External Key Switching

Proof. Let $\{b_k\}_{k=0}^{n-1} \subset \hat{\mathcal{R}}_q$ be the ciphertexts produced by modulus switching, whose constant terms satisfy $\text{ct}(b_k f_Q) = e_{k,0} + \Delta \cdot u_{k,0}$ with $m_k = u_{k,0}$, where $e_{k,0}$ denotes the constant term of $\text{err}(b_k)$. With P_i and $\mathbf{ksk}_i^{\text{ext}}$ as defined in Section 4.2, multiplying the output $c' = \sum_{i,l} \mathbf{ksk}_{i,l}^{\text{ext}} \cdot \mathbf{g}^{-1}(P_i)_l$ by the target key f and using $\sum_l \mathbf{g}^{-1}(P_i)_l \cdot B^l = P_i$ gives

$$c' \cdot f = \underbrace{\sum_{i,l} h_{i,l} \mathbf{g}^{-1}(P_i)_l}_{=: g_h} + \sum_{i=0}^{N-1} s_i P_i = g_h + \Phi(X),$$

where the last equality is the regrouping identity (4.2). Substituting $\text{ct}(b_k f_Q) = e_{k,0} + \Delta u_{k,0}$ into $\Phi(X) = \sum_k \text{ct}(b_k f_Q) X^k$ yields $\Phi(X) = \sum_k e_{k,0} X^k + \Delta \cdot m$; hence $c' = (g' + \Delta \cdot m) \cdot f^{-1} \in \mathcal{R}_q$ with $g' := g_h + \sum_k e_{k,0} X^k$ is a valid encryption of m under the base key f , provided g' remains within the decryption tolerance. $\square$

References

- [ABD16] Martin R. Albrecht, Shi Bai, and Léo Ducas. A subfield lattice attack on over-stretched NTRU assumptions - cryptanalysis of some FHE and graded encoding schemes. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 153–178. Springer, Berlin, Heidelberg, August 2016.
- [BIP⁺22] Charlotte Bonte, Ilia Iliashenko, Jeongeun Park, Hilder V. L. Pereira, and Nigel P. Smart. FINAL: Faster FHE instantiated with NTRU and LWE. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 188–215. Springer, Cham, December 2022.
- [BKS⁺21] Fabian Boemer, Sejun Kim, Gelila Seifu, Fillipe D. M. de Souza, and Vinodh Gopal. Intel HEXL: Accelerating homomorphic encryption with intel AVX512-IFMA52. Cryptology ePrint Archive, Report 2021/420, 2021.
- [BLLN13] Joppe W. Bos, Kristin Lauter, Jake Loftus, and Michael Naehrig. Improved security for a ring-based fully homomorphic encryption scheme. In Martijn Stam, editor, *14th IMA International Conference on Cryptography and Coding*, volume 8308 of *LNCS*, pages 45–64. Springer, Berlin, Heidelberg, December 2013.
- [CGGI16] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part I*, volume 10031 of *LNCS*, pages 3–33. Springer, Berlin, Heidelberg, December 2016.
- [CHHS19] Jung Hee Cheon, Minki Hhan, Seungwan Hong, and Yongha Son. A hybrid of dual and meet-in-the-middle attack on sparse and ternary secret LWE. Cryptology ePrint Archive, Report 2019/1114, 2019.
- [DM15] Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 617–640. Springer, Berlin, Heidelberg, April 2015.
- [DvW21] Léo Ducas and Wessel P. J. van Woerden. NTRU fatigue: How stretched is over-stretched? In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 3–32. Springer, Cham, December 2021.
- [GP25] Antonio Guimarães and Hilder V. L. Pereira. Fast amortized bootstrapping with small keys and polynomial noise overhead. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 2967–2981. ACM Press, October 2025.
- [HPS98] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU: A ring-based public key cryptosystem. In *Third Algorithmic Number Theory Symposium (ANTS)*, volume 1423 of *LNCS*, pages 267–288. Springer, June 1998.
- [KF17] Paul Kirchner and Pierre-Alain Fouque. Revisiting lattice attacks on over-stretched NTRU parameters. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part I*, volume 10210 of *LNCS*, pages 3–26. Springer, Cham, April / May 2017.

- [LTV12] Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In Howard J. Karloff and Toniann Pitassi, editors, *44th ACM STOC*, pages 1219–1234. ACM Press, May 2012.
- [Zam22] Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. <https://github.com/zama-ai/tfhe-rs>.